COMSATS University Islamabad, Lahore

Mid exam – Fall 2025

Course Title:	Game Development	Course Code:	CSC 495	Credit Hours:	3(2,1)
Course Instructor/s:	M. Mohsin Mehdi	Programme Name:	BS Computer Science		
Semester:	Batch:	Section: A,B, C	Date:		
Time Allowed:	**90 mins**	**Maximum Marks:**		**30**	
Student's Name:		Reg. No.	CIIT/ - - - /LHR		

Q1. (20 Marks) CLO-1<Understanding>Characterize the fundamental concepts of game development.
Answer all parts. Each part carries 2 marks. Write your answers in the space provided.

a. Both scripts are attached to the same GameObject. What will be printed in the Console, and why?

```
public class A : MonoBehaviour {
   void Awake() { Debug.Log("A Awake"); }
   void Start() { Debug.Log("A Start"); }
}
public class B : MonoBehaviour {
   void Start() { Debug.Log("B Start"); }
   void Awake() { Debug.Log("B Awake"); }
}
```

SOLUTION:
Output:
A Awake
B Awake
A Start
B Start

Explanation: In Unity, Awake() is called for all scripts before any Start() methods. The order of Awake() calls is not guaranteed between scripts, but both Awake() methods complete before any Start() methods begin. Then Start() methods are called, again in an undefined order between scripts.

b. Predict the final position of the cube in world space and explain why.

```
void Start() {
   GameObject parent = new GameObject("Parent");
   GameObject cube = GameObject.CreatePrimitive(PrimitiveType.Cube);
   cube.transform.parent = parent.transform;
   parent.transform.position = new Vector3(5, 0, 0);
   cube.transform.localPosition = new Vector3(2, 0, 0);
   Debug.Log(cube.transform.position);
```

SOLUTION:
Final position: (7, 0, 0) in world space

Explanation: When a child object's localPosition is set, it's relative to the parent. The parent is at world position (5, 0, 0). The cube's localPosition is (2, 0, 0), so its world position is parent position + local position = (5, 0, 0) + (2, 0, 0) = (7, 0, 0).
}

c. The same script is attached to a GameObject in a scene. When the scene reloads using SceneManager.LoadScene(0);, what will be printed and why?

```
public class Counter : MonoBehaviour {
   public static int value = 0;
   void Start() {
      value++;
      Debug.Log(value);
   }
```

SOLUTION:

Output: The value will continue incrementing (1, 2, 3, etc.) on each scene reload.

Explanation: Static variables persist across scene loads in Unity. When SceneManager.LoadScene(0) reloads the scene, the static variable 'value' is not reset to 0. Each time Start() is called, it increments the existing value.
}

d. Why will the object accelerate infinitely and jitter? Rewrite the correct version that uses Unity' s physics system properly.

```
void Update() {
   Rigidbody rb = GetComponent<Rigidbody>();
   rb.AddForce(Vector3.forward * 10f);
```

SOLUTION:

The object will accelerate infinitely because AddForce is called every frame in Update(), continuously adding force. It will jitter because Update() is frame-rate dependent and physics calculations happen at fixed intervals, causing inconsistent force application.

Correct version:

```
void FixedUpdate() {
   Rigidbody rb = GetComponent<Rigidbody>();
   rb.AddForce(Vector3.forward * 10f);
}
```

Explanation: Use FixedUpdate() instead of Update() for physics operations. FixedUpdate() runs at fixed intervals (default 50Hz) and is synchronized with the physics engine, preventing jitter and ensuring consistent force application.
}

e. What happens when you press the Space key twice? Explain what prints and why.

```
IEnumerator Start() {
```

```
    StartCoroutine(PrintMessage());
    yield return null;
}
IEnumerator PrintMessage() {
    Debug.Log("Running");
    yield return new WaitForSeconds(2);
    Debug.Log("Done");
}
void Update() {
    if (Input.GetKeyDown(KeyCode.Space))
        StopCoroutine(PrintMessage());
```

SOLUTION:

Pressing Space twice will not stop the coroutine on the second press.

Explanation: When you press Space the first time, StopCoroutine(PrintMessage()) is called, but it doesn't stop the coroutine because you need a reference to the coroutine instance. The coroutine started in Start() is not stored, so StopCoroutine() cannot find it to stop. The coroutine will continue running and print "Running" and "Done" after 2 seconds regardless of Space key presses.

To fix, store the coroutine reference:

```
Coroutine coroutineRef;

IEnumerator Start() {
    coroutineRef = StartCoroutine(PrintMessage());
    yield return null;
}

void Update() {
    if (Input.GetKeyDown(KeyCode.Space) && coroutineRef != null)
        StopCoroutine(coroutineRef);
}
}
```

f. The message does not appear even though the object’ s tag is *Enemy*. Explain and correct the issue.

```
void OnTriggerEnter(Collider other) {
    if (other.tag == "Enemy")
        Debug.Log("Hit Enemy");
```

SOLUTION:

The message does not appear because OnTriggerEnter requires at least one of the colliding objects to have a Rigidbody component, and the collider must be marked as "Is Trigger" in the Inspector.

Corrected version:

1. Add a Rigidbody component to the GameObject with this script (or the other object)
2. Ensure the Collider component has "Is Trigger" checked
3. The code itself is correct, but the setup is missing

Alternative: If using OnCollisionEnter instead:

```
void OnCollisionEnter(Collision collision) {
   if (collision.gameObject.tag == "Enemy")
      Debug.Log("Hit Enemy");
}
}
```

g. The ray always misses the target even though it is directly in front of the object. Identify the logical mistake and provide the corrected version.

```
void Update() {
 RaycastHit hit;
if (Physics.Raycast(transform.position, transform.TransformDirection(Vector3.forward), out hit, 100f));
Debug.Log(hit.collider.name);
}
```

SOLUTION:

The semicolon is the logical error.

h. Will the destroyed object’ s name still print? Explain Unity’ s destruction timing rules.

```
void OnTriggerEnter(Collider other) {
   Destroy(other.gameObject);
   Debug.Log(other.name);
```

SOLUTION:

Yes, the destroyed object's name will still print.

Explanation: In Unity, Destroy() does not immediately remove the object. It marks the object for destruction at the end of the current frame, after all Update(), LateUpdate(), and other frame events complete. Therefore, other.gameObject and other.name are still valid and accessible immediately after Destroy() is called. The object will be destroyed after the current frame ends.

```
}
```

i. What visible issue will occur in this script and why?

```
void Update() {
   transform.Rotate(Vector3.up * 90f * Time.deltaTime * Time.deltaTime);
```

SOLUTION:

The rotation will appear to slow down over time and eventually stop.

Explanation: The issue is using Time.deltaTime twice. Time.deltaTime is already a small fraction (typically 0.016 for 60fps). Multiplying it by itself (Time.deltaTime * Time.deltaTime) makes it extremely small (0.000256), causing the rotation to be almost imperceptible and appear to slow down dramatically. The rotation speed decreases quadratically with frame rate.

Correct version:

```
void Update() {
   transform.Rotate(Vector3.up * 90f * Time.deltaTime);
}
}
```

j. Predict the printed order of letters and explain why.

```
IEnumerator Start() {
   Debug.Log("A");
   yield return new WaitForSeconds(1);
   Debug.Log("B");
}
void Awake() {
   Debug.Log("C");
}
void OnEnable() {
   Debug.Log("D");
```

SOLUTION:

Printed order: D, C, A, B

Explanation: Unity's execution order for MonoBehaviour lifecycle methods:
1. OnEnable() - called when object becomes enabled (D)
2. Awake() - called when script instance is loaded (C)
3. Start() - called before first frame update (A)
4. Coroutine continues after yield - WaitForSeconds(1) then prints B

The coroutine in Start() yields immediately (yield return null), so "A" prints, then execution continues to the next frame. After 1 second, "B" is printed.

```
}
```

Q2. (5 Marks) CLO 3<Creating>Create animations for a game scenario.

How can you make sure that the animation does not depend on frame rate in update function?

SOLUTION:

Animation called by events of button click or mouse click does not depend on frame rate.

Q3. (5 Marks) CLO 2 <Creating> Create different assets and scenes for a game scenario.

Describe the process of **scene transition** in Unity using SceneManager.LoadScene(). What happens to GameObjects, static variables, and coroutines when a scene reloads? How can specific data (like score or player stats) be preserved across scenes?

SOLUTION:

Process of scene transition using SceneManager.LoadScene():

1. Call SceneManager.LoadScene(sceneIndex) or SceneManager.LoadScene(sceneName)
2. Unity unloads the current scene

3. Unity loads the new scene
4. All GameObjects in the old scene are destroyed
5. All scripts in the old scene are destroyed
6. Static variables PERSIST across scene loads (they are not reset)
7. All coroutines are stopped and destroyed with their GameObjects
8. New scene's GameObjects are instantiated
9. Awake() and OnEnable() are called for new objects
10. Start() is called for new objects

What happens to:
- GameObjects: All GameObjects in the previous scene are destroyed. Only GameObjects marked as "DontDestroyOnLoad" persist.
- Static variables: They persist across scene loads. They are not reset unless explicitly set to a new value.
- Coroutines: All coroutines are stopped when their GameObjects are destroyed. Coroutines do not persist across scene loads.